

Check CCTV Insights Code On Github

<https://github.com/campusinsights24/CCTV-Insights/blob/main/InsightsCCTV.py>

Here I Teardown Every Step Of Code

It 100% Works With Your Google Drive

No Any Server Or Third Party App Trace Your Activities

It is only compatible to works With Windows 10/11

1. Library Imports

कोड की शुरुआत में बाहरी टूल्स (Libraries) को इम्पोर्ट किया गया है:

- **os और sys**: फाइल्स/फोल्डर बनाने, पाथ (Path) सेट करने और सिस्टम प्रोसेस को बंद करने के लिए।
- **cv2 (OpenCV)**: यह कंप्यूटर विजन (Computer Vision) लाइब्रेरी है, जो Webcam चालू करती है और वीडियो रिकॉर्ड करती है।
- **time और datetime**: समय ट्रैक करने, वीडियो/फोटो फाइल के नाम में डेट और टाइम स्टैम्प (Timestamp) लगाने के लिए।
- **glob**: कंप्यूटर के अंदर सभी पुरानी .mp4 फाइल्स को एक साथ ढूँढने (Search) के लिए।

- **threading**: एक ही समय में कई काम (Multithreading) करने के लिए। (जैसे: एक तरफ वीडियो रिकॉर्डिंग, दूसरी तरफ कीबोर्ड/माउस ट्रैकिंग)।
- **subprocess**: बैकग्राउंड में छुपे हुए कमांड्स (जैसे Ping) रन करने के लिए।
- **tkinter (tk)**: स्क्रीन पर दिखने वाला डिज़ाइन यानी Graphical User Interface (GUI) बनाने के लिए।
- **numpy (np)**: डेटा एरे (Array) और मैट्रिक्स (Matrix) कैलकुलेशन के लिए (अगर कैमरा फ्रेम मिस कर दे, तो ब्लॉक ब्लॉक स्क्रीन जनरेट करने के लिए)।
- **ctypes**: विंडोज के कोर सिस्टम लेवल (C-level) API से बात करने के लिए (लैपटॉप के चार्जर/बैटरी का स्टेटस जानने के लिए)।

try... except ImportError **ब्लॉक**:

- यहाँ win32api, win32con, pythoncom, win32com.client को इम्पोर्ट किया गया है। ये Windows OS के सेंसर्स (Sensors) हैं जो माउस, कीबोर्ड और USB की जानकारी देते हैं।
- अगर आपके सिस्टम में यह इंस्टॉल नहीं है (ImportError), तो कोड क्रैश होने के बजाय "CRITICAL" मैसेज प्रिंट करता है और sys.exit(1) से Safely Program को रोक देता है।

INSTA @campusinsights24

7988407499

2. UIOverlay Class (यूज़र इंटरफ़ेस और सिक्योरिटी लॉक)

यह पूरा ब्लॉक स्क्रीन पर दिखने वाले 'Terminal' डिज़ाइन और सिक्योरिटी पिन को हैंडल करता है।

- **__init__(self, dvr_engine)**: यह क्लास का स्टार्टर है। यह CCTVInsightsDVR (कैमरा इंजन) को UI के साथ जोड़ता है और setup_window() तथा build_ui() फंक्शन को कॉल करता है।
- **setup_window(self)**:
 - self.app_name: पाइथन फाइल का नाम निकालकर उसे ऐप का टाइटल बनाता है।
 - यह 850x550 पिक्सल की एक स्क्रीन बनाता है और उसे स्क्रीन के बिल्कुल सेंटर (Center) में सेट करता है।

- `bg="#050505"`: बैकग्राउंड कलर डार्क ब्लैक सेट करता है।
- `attributes("-topmost", True)`: इस विंडो को हमेशा दूसरी सभी ऐप्स के ऊपर (Always on top) रखता है।
- `overrideredirect(True)`: विंडोज के डिफ़ॉल्ट टाइटल बार (जिसमें X बटन होता है) को हटा देता है, ताकि ऐप बॉर्डरलेस (Borderless) दिखे।
- `on_window_restore(self, event)` और `minimize_app(self)`: जब विंडो मिनिमाइज़ (Minimize) की जाती है, तो बॉर्डरलेस इफ़ेक्ट हटाकर उसे टास्कबार (Taskbar) में भेजता है, और वापस आने पर फिर से बॉर्डरलेस कर देता है।
- `close_app_attempt(self)`:
 - अगर सिस्टम **Armed** (रिकॉर्डिंग ऑन) है और कोई 'X' बटन दबाता है, तो यह ऐप बंद नहीं होने देता। यह लाल रंग में एरर देता है: [ACTION BLOCKED: ENTER PIN TO DISARM & EXIT]।
 - अगर सिस्टम **Unarmed** है, तो यह `force_hardware_shutdown()` को कॉल करके ऐप बंद कर देता है।
- `drag_start` और `drag_motion`: कस्टम टाइटल बार को पकड़ कर माउस से खिसकाने (Drag/Drop) का लॉजिक।
- `resize_start` और `resize_motion`: स्क्रीन के राइट बॉटम (Right Bottom) कोने को पकड़ कर विंडो का साइज़ (Resize) बड़ा-छोटा करने का लॉजिक। इसमें मिनिमम लिमिट 750x500 सेट है।
- `build_ui(self)`: यहाँ स्क्रीन के अंदर के सारे एलिमेंट्स (Elements) डिज़ाइन किए गए हैं:
 - एक कस्टम `title_bar` बनाया गया है जिसमें 'X' (Close) और '-' (Minimize) बटन हैं। उन पर Hover Effect (माउस ले जाने पर कलर बदलना) लगाया गया है।
 - स्क्रीन पर "CAMPUS INSIGHTS" और फीचर्स (UG/PG Courses आदि) `tk.Label` की मदद से प्रिंट किए गए हैं।
 - `pin_entry`: एक पासवर्ड बॉक्स बनाया गया है जहाँ इनपुट को * (Asterisk) के रूप में छुपाया गया है।
 - `btn_toggle`: यह 'SHOW' और 'HIDE' बटन है जो पासवर्ड दिखाता या छुपाता है।

- `self.root.bind('<Return>', self.process_pin_action)`: कीबोर्ड के 'Enter' बटन को प्रोसेस पिन से लिंक किया गया है।
- `toggle_pin(self)`: यह चेक करता है कि अगर बॉक्स में `show='*'` है तो उसे हटा दो, और अगर नहीं है तो लगा दो।
- `process_pin_action(self)`: (कोर सिक््योरिटी लॉजिक)
 - `if not self.dvr.is_armed`: (जब सिस्टम बंद है): यूज़र को कम से कम 4 अंकों का पिन डालना होगा। वह पिन `self.dvr.master_pin` बन जाता है। बॉक्स खाली हो जाता है और सिस्टम **ARMED** हो जाता है। `self.dvr.arm_system()` कॉल होकर रिकॉर्डिंग शुरू हो जाती है।
 - `else`: (जब सिस्टम चालू है): अगर कोई इसे बंद करना चाहता है, तो उसे वही पुराना मास्टर पिन डालना होगा। मैच होने पर सिस्टम **DISARMED** होता है और ऐप बंद हो जाती है। गलत पिन होने पर लाल एरर आता है।
- `force_hardware_shutdown(self)`: यह `root.destroy()` से UI हटाता है और `os._exit(0)` से बैकग्राउंड पाइथन प्रोसेस को बेरहमी से (Forcefully) किल करता है, ताकि वेबकैम तुरंत फ्री हो जाए।
- `run(self)`: `mainloop()` चलाता है जो इस UI को स्क्रीन पर लगातार दिखाता रहता है।

 INSTA @campusinsights24

7988407499

3. CCTVInsightsDVR Class (कोर कैमरा और सेंसर इंजन)

यह क्लास सिक््योरिटी सिस्टम का बैकएंड (Backend) है।

- `__init__(self)`:
 - यहाँ बेसिक वेरिएबल्स सेट हैं: `is_armed = False`, फ्रेम रेट `fps = 5.0`, और एक वीडियो की लेंथ `chunk_duration = 60.0` (1 मिनट)।
 - `self.frame_lock = threading.Lock()`: यह थ्रेड्स के बीच फ्रेम को क्रैश होने से बचाता है।
 - `self.app_name`: पाइथन फाइल का नाम निकालता है।

- **Smart Google Drive Auto-Locator:** एक लूप चलता है जो C, D, E, F, G ड्राइव में जाकर 'My Drive' (गूगल ड्राइव) खोजता है। मिल जाए तो ठीक, नहीं तो डेस्कटॉप (Desktop) का पाथ चुन लेता है।
- `os.makedirs`: जो भी पाथ मिला, वहाँ पाइथन फाइल के नाम का एक मेन फोल्डर बना देता है।
- `win32api.SetConsoleCtrlHandler`: विंडोज के शटडाउन इवेंट्स (Shutdown Events) को मॉनिटर करने के लिए अटैच किया गया है।
- **`arm_system(self)`:**
 - पिन सही डलने पर यह कॉल होता है।
 - करंट डेट और टाइम (`Session_Date_Time`) का एक नया फोल्डर बनाता है।
 - उसके अंदर `INTRUDER_SNAPSHOTS` नाम का सब-फोल्डर (Sub-folder) बनाता है।
 - इसके बाद यह 3 बैकग्राउंड थ्रेड (Daemon Threads) चालू कर देता है: नेटवर्क मॉनिटर, मेन कैमरा इंजन, और हार्डवेयर मॉनिटर।
- **`_os_shutdown_handler(self, ctrl_type)`:**
 - अगर कोई चोर डायरेक्ट कंप्यूटर को शटडाउन या लॉग ऑफ (Logoff) करता है, तो विंडोज इसे ट्रिगर करता है। यह तुरंत `_take_snapshot_burst()` को रन कर देता है।
- **`_safe_image_write(self, path, frame)`:**
 - यह OpenCV का `imencode` यूज़ करता है ताकि फोटो सेव करते समय अगर हार्ड डिस्क बिज़ी हो, तो फाइल करप्ट (Corrupt) न हो।
- **`_take_snapshot_burst(self)`:**
 - शटडाउन के समय यह लूप 5 बार चलता है। हर 1 सेकंड में वेबकैम के करंट फ्रेम (`latest_frame`) की 5 फोटो `Intruder_Shutdown_...jpg` नाम से खींच लेता है।
- **`_hardware_activity_monitor(self)`: (घुसपैठिया अलार्म सिस्टम)**
 - `pythoncom.ColInitialize()`: थ्रेडिंग में COM ऑब्जेक्ट्स यूज़ करने के लिए ज़रूरी है।
 - `SYSTEM_POWER_STATUS`: बैटरी और पावर प्लग का स्टेटस पकड़ने के लिए सी-टाइप (C-type) स्ट्रक्चर बनाता है।

- शुरुआती पोजीशन कैप्चर करता है: माउस (last_mouse), पावर (last_power), और USB (last_usb_count) WMI क्वेरी के थ्रू।
- **While Loop (self.is_armed):** यह लूप लगातार चलता रहता है:
 - **Mouse Check:** करंट और लास्ट माउस X/Y कोऑर्डिनेट्स (Coordinates) में 30 पिक्सल से ज़्यादा का डिफ़रेंस चेक करता है।
 - **Keyboard Check:** 8 से 256 तक के सभी कीबोर्ड बटन (win32api.GetAsyncKeyState) को स्कैन करता है कि कोई बटन दबाया गया या नहीं।
 - **Power Check:** चार्जर निकाला या लगाया गया या नहीं (power_status.ACLineStatus)।
 - **USB Check:** हर 10वें लूप साइकिल में चेक करता है कि कोई नई पेनड्राइव कनेक्ट हुई है या नहीं।
- **10-Second Threat Burst Rule:** अगर ऊपर से कोई भी मूवमेंट (triggered_now == True) डिटेक्ट होती है, तो यह अलार्म विंडो को अगले 10 सेकंड (alert_end_time = time.time() + 10.0) तक बढ़ा देता है।
- अलार्म विंडो के दौरान, यह 1-1 सेकंड के गैप पर Snap_Date_Time.jpg नाम से फोटो सेव करता रहता है।
- **execute_global_fifo_cleanup(self): (स्टोरेज क्लीनर)**
 - MAX_CAPACITY = 12 * 1024 * 1024 * 1024 (12 GB का मैक्स लिमिट)।
 - glob के थ्रू मेन फोल्डर के अंदर के सारे .mp4 फाइल्स की लिस्ट बनाता है और उन्हें पुराने से नए डेट के हिसाब से सॉर्ट (Sort) करता है।
 - while total_size > MAX_CAPACITY: जब तक फोल्डर 12 GB से ऊपर है, यह सबसे पुरानी वीडियो फाइल (pop(0)) को डिलीट (os.remove) करता रहता है।
- **_network_sentineLdaemon(self):**
 - यह हर 10 सेकंड में बैकग्राउंड में (बिना कोई विंडो खोले CREATE_NO_WINDOW) गूगल (8.8.8.8) को पिंग (Ping) करता है।
- **core_dvr_engine(self): (वीडियो रिकॉर्डिंग लूप)**

- `cv2.VideoCapture(0, cv2.CAP_DSHOW)`: कैमरा चालू करता है (DSHOW विंडोज में कैमरे को फ़ास्ट चालू करता है)।
- रेज़ोल्यूशन (Resolution) 640x480 सेट करता है।
- `target_frames`: (60 सेकंड * 5 FPS = 300 फ्रेम्स) प्रति वीडियो कैलकुलेट करता है।
- `while self.is_armed`:
 - करंट डेट-टाइम से फाइल का नाम बनाता है (`Insights_CCTV_...mp4`)।
 - `VideoWriter` ऑब्जेक्ट बनाता है जो `mp4v` कोडेक (Codec) यूज़ करता है।
 - जब तक `frames_processed` लिमिट (300) तक नहीं पहुँचता, यह `cap.read()` से फोटो खींचता है।
 - `latest_frame` को मेमोरी में लॉक (`frame_lock`) करता है (ताकि मॉनिटर थ्रेड सैपशॉट खींच सके)।
 - फ्रेम को वीडियो फाइल में राइट (`write`) करता है। (कैमरा हैंग हो जाए तो ब्लैक फ्रेम राइट करता है)।
 - टाइम स्लीप (`time.sleep`) कैलकुलेट करता है ताकि वीडियो बिल्कुल एक्जैक्ट 5 FPS पर ही चले।
 - वीडियो लिमिट पूरी होने पर फाइल को रिलीज़ (`release`) करता है और तुरंत बैकग्राउंड में `execute_global_fifo_cleanup` थ्रेड को कॉल कर देता है।
- अंत में जब सिस्टम Disarm होता है, तो `self.cap.release()` से कैमरे का हार्डवेयर कंट्रोल छोड़ देता है।

4. Execution Block (मेन स्टार्टर)

- `if __name__ == "__main__":`
 - यह पाइथन का एंट्री पॉइंट (Entry Point) है। जब फाइल रन होती है, तो कोड सीधा यहाँ आता है।

- `dvr = CCTVInsightsDVR()`: यह कैमरे और सेंसर इंजन का ऑब्जेक्ट (Object) मेमोरी में लोड करता है।
- `app = UIOverlay(dvr)`: यह UI का ऑब्जेक्ट लोड करता है और बैकएंड (dvr) को UI के साथ कनेक्ट कर देता है।
- `app.run()`: स्क्रीन पर UI को डिस्प्ले (Display) कर देता है और आपका सॉफ्टवेयर चालू हो जाता है।

